CIS*3110 Operating Systems

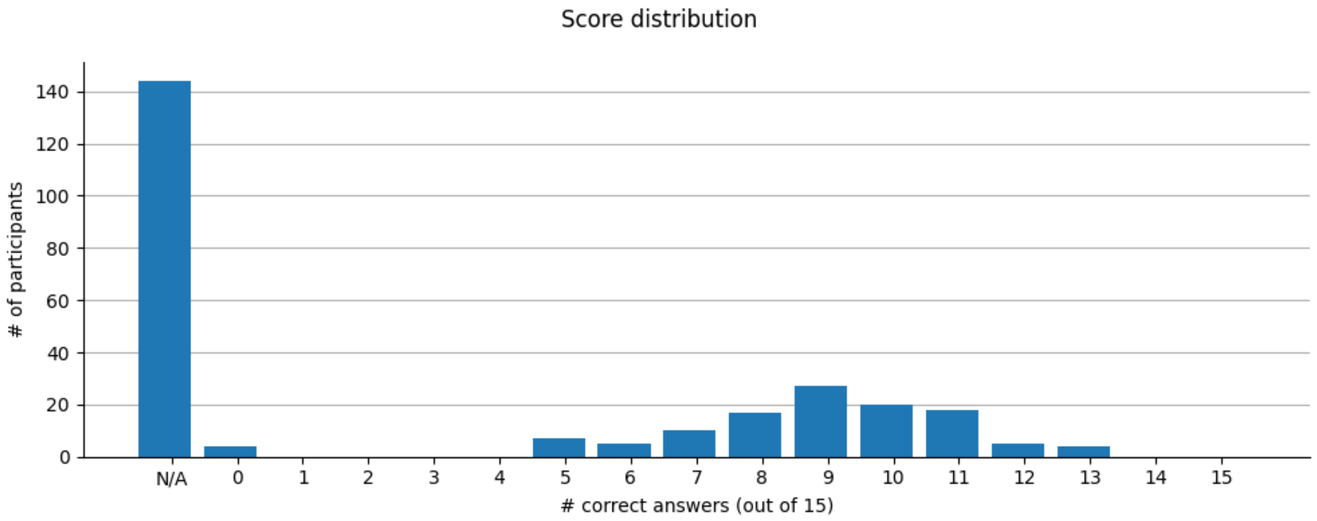
Week 9
Lecture #13 Threads and locks

Prof. Yulia Kotseruba
March 5, 2026

Quiz 12

the mean will go up once I remove 2 questions

- **61%** average (up from 59% in quiz 11)
- 3 hardest questions:
  - (30%) A CFS scheduler calculates vruntime of the process based on its ____ a) CPU share b) nice value c) current vruntime d) all of the above
  - (56%) 100, 200, and 150 KB are allocated contiguously, then 100 and 150 KB are freed. When 250 KB request arrives _____ occurs. a) no problem b) internal fragmentation c) external fragmentation d) page fault
  - (58%) What is a major disadvantage of swapping? a) uses too much CPU b) slow due to I/O c) fragmentation d) TLB misses



Recap

- Each process has **one or more threads**
- Threads are **lightweight processes**
  - share heap, data, and code
  - have their own stack, program counter, registers, thread id
- pthreads provides a flexible API for managing threads
  - `pthread_create(), pthread_join(), pthread_detach(), pthread_exit()`
- Avoid common pitfalls
  - applying join and detach operations on the same thread
  - joining/detaching the same thread twice
  - returning local thread variables to main

Recap

	Process	Thread
Description	Is a program that is executed by one or more threads	A basic unit of CPU computation
Resource sharing	No sharing of heap, data, stack, registers (some processes may share code)	Share heap, code, data Own stack, PC, id and registers
Address space	Separate	Shared
Communication	Interprocess communication, e.g. pipes/ sockets	Via global variable
Context switching	Slower (need to save/load the entire address space)	Faster (need to update the state)
Parallelism	Each process is scheduled and run independently of other processes	Depends on the multithreading model

Recap

- Pros of threads: shared memory
  - **quicker** to create a thread
    - no need to allocate memory, no cache misses, etc.
  - **faster** context switching
    - less stuff to move around (only registers and PC)
  - **easier** data sharing
    - exchange data/messages without special mechanisms
  - **parallel** execution
    - can wait for I/O and do useful work simultaneously
- Cons of threads: shared memory
    - **no security** among threads

Threads

- Why bother with threads when we already have processes?
  - Reason #1 Better parallelism
  - Reason #2 Reducing overhead

Parallelism

- Task - a unit of computation that can be executed in parallel with other tasks on multiple CPU cores/processors
- Data parallelism
  - performing the **same task** on **different** subsets of **data** in parallel
  - e.g. matrix multiplication

Parallelism

- Task - a unit of computation that can be executed in parallel with other tasks on multiple CPU cores/processors
- Data parallelism
  - performing the **same task** on **different** subsets of **data** in parallel
  - e.g. matrix multiplication

multiplying matrices

A B AxB = C

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

$$c_1 = a_1 * b_1 + a_2 * b_4 + a_3 * b_7$$

each entry in matrix C can be computed independently, i.e. we do not need to know the value of c2 to compute c1

we can compute C in any order and in parallel

Parallelism

- Task - a unit of computation that can be executed in parallel with other tasks on multiple CPU cores/processors
- Data parallelism
  - performing the **same task** on **different** subsets of **data** in parallel
  - e.g. matrix multiplication
- Task parallelism
  - performing **different tasks** in parallel
  - e.g. one thread does computation and another does I/O

multiplying matrices

A B AxB = C

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

$$c_1 = a_1 * b_1 + a_2 * b_4 + a_3 * b_7$$

each entry in matrix C can be computed independently, i.e. we do not need to know the value of c2 to compute c1

we can compute C in any order and in parallel

Overhead

- Threads share heap, data and code
  - thread creation/destruction is **cheaper**
    - new thread will need only a new stack and registers
    - **no system calls** needed, e.g. to allocate/deallocate memory
  - context switching is **faster**
    - only need to update the thread's **state** (program counter and registers)
  - communication is **easier**
    - threads can communicate through **global variables**
    - no need for setting up pipes/sockets

Types of threads

- **kernel threads**
  - the **real** threads that execute on the CPU
  - are created and managed by the OS kernel
- **user-level threads**
  - are created and managed by a **user-level thread library**
  - they do not execute on the CPU on their own
  - must be **mapped** to a kernel thread
- kernel-only threads
  - execute only in kernel mode environment

Kernel threads

- Created and managed by the OS
- Can run on different CPU processors/cores
- Managed by the CPU scheduler

Kernel threads

- Created and managed by the OS
- Can run on different CPU processors/cores
- Managed by the CPU scheduler

- Aside: we only looked at **uniprocessor scheduling** with **single-threaded processes**
  - most computers these days have multiple processors/cores
  - most applications are multi-threaded
  - this adds complexity to scheduling, e.g.
    - how to distribute threads across processors/cores?
    - should threads of the same process be treated individually or as a group?

Chapter 10 on multiprocessor scheduling

User threads

- Created and managed by the **user-level thread library**
- **No system calls** are needed to create them
- Unseen by the kernel unless they are **mapped** (attached) to the kernel thread

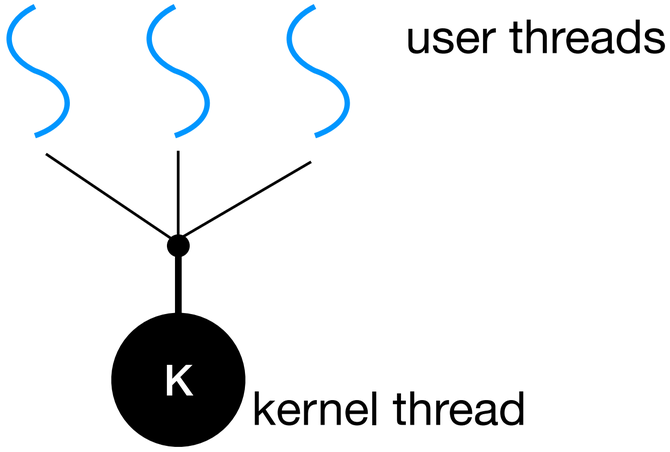
- Aside: is pthread a user-level thread library?
  - No, because pthread is an API
  - pthread prescribes behaviors
  - implementations are OS-specific

Multithreading models

- Different ways of mapping user threads to kernel threads
  - **many-to-one** - many user-level threads are mapped onto 1 kernel thread
  - **one-to-one** - separate kernel thread to handle each user thread
  - **many-to-many** - any number of user threads onto <= kernel threads

Multithreading models

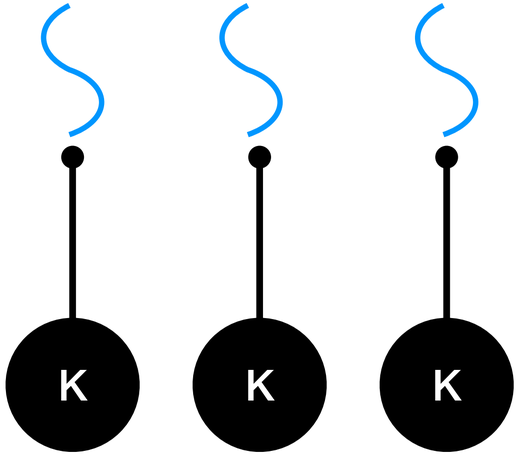
one-to-many - many user threads map onto one kernel thread



pros: efficiency - threads are managed in the user space

cons: threads are treated as a group
- if one blocks, all are stuck
- cannot run on multiple CPUs

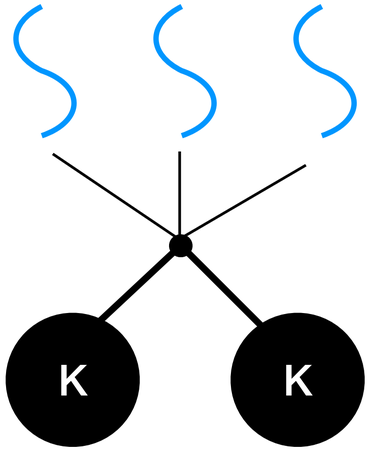
one-to-one - each user thread maps onto one kernel thread



pros: threads are treated individually
- scheduled individually
- can run on multiple CPUs

cons: more overhead, limit on the number of threads possible

many-to-many - N user threads are mapped to M $\leq$ N kernel threads



pros: flexibility - threads are treated individually or as a group depending on the number of CPUs

cons: complexity of implementation

Multithreading models

- Different ways of mapping user threads to kernel threads
  - **many-to-one** - many user-level threads are mapped onto 1 kernel thread
    - pros: fast and easy to implement
    - cons: no parallelism
    - old Linux kernels implemented
  - **one-to-one** - separate kernel thread to handle each user thread
    - pros: full parallelism
    - cons: overhead
    - modern default (Linux, mac OS, Windows)
    - heavy investment into reducing the overhead
  - **many-to-many** - any number of user threads onto <= kernel threads
    - pros: flexibility
    - cons: complexity
    - largely abandoned, not worth the benefit

```c
// includes omitted
void *worker(void *arg) {
  printf("[thread] %s\n", (char *) arg);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p1, p2;
  int rc;
  printf("[main] begin\n");
  pthread_create(&p1, NULL, worker, "A");
  pthread_create(&p2, NULL, worker, "B");
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] end\n");
  return 0;
}
```

thread_print.c

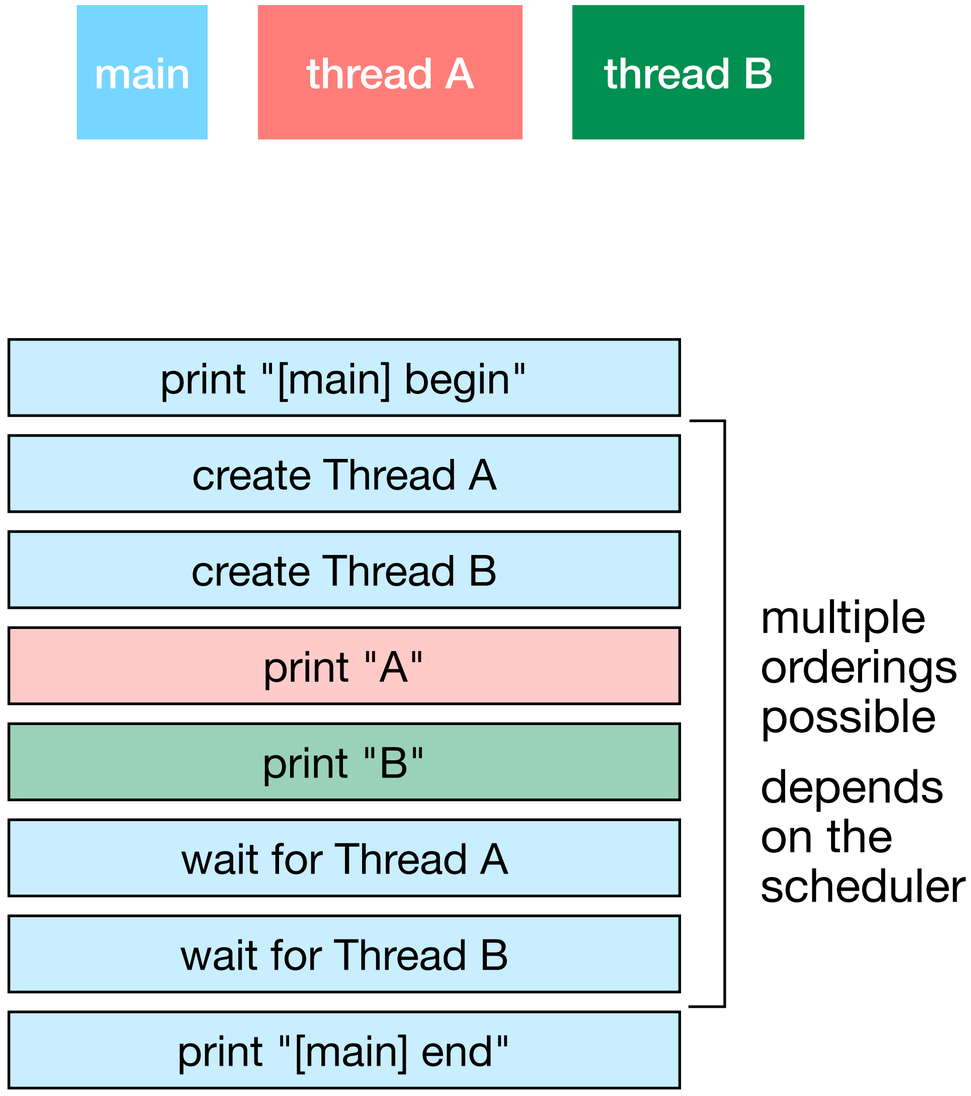
```
>>> ./thread_print
[main] begin
[thread] A
[thread] B
[main] end
```

```
>>> ./thread_print
[main] begin
[thread] B
[thread] A
[main] end
```

A and B will be printed but the order depends on scheduling and cannot be determined in advance

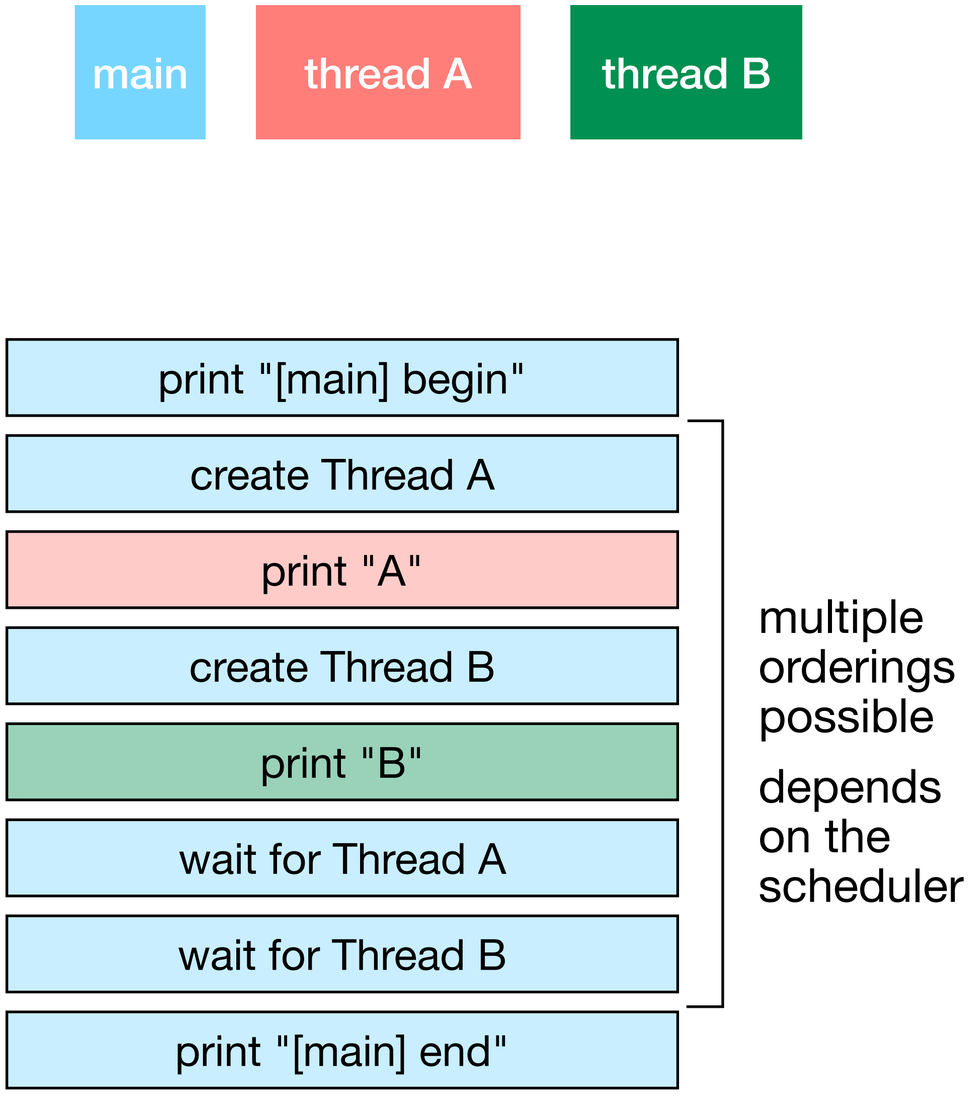
```c
// includes omitted
void *worker(void *arg) {
  printf("[thread] %s\n", (char *) arg);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p1, p2;
  int rc;
  printf("[main] begin\n");
  pthread_create(&p1, NULL, worker, "A");
  pthread_create(&p2, NULL, worker, "B");
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] end\n");
  return 0;
}
```

thread_print.c



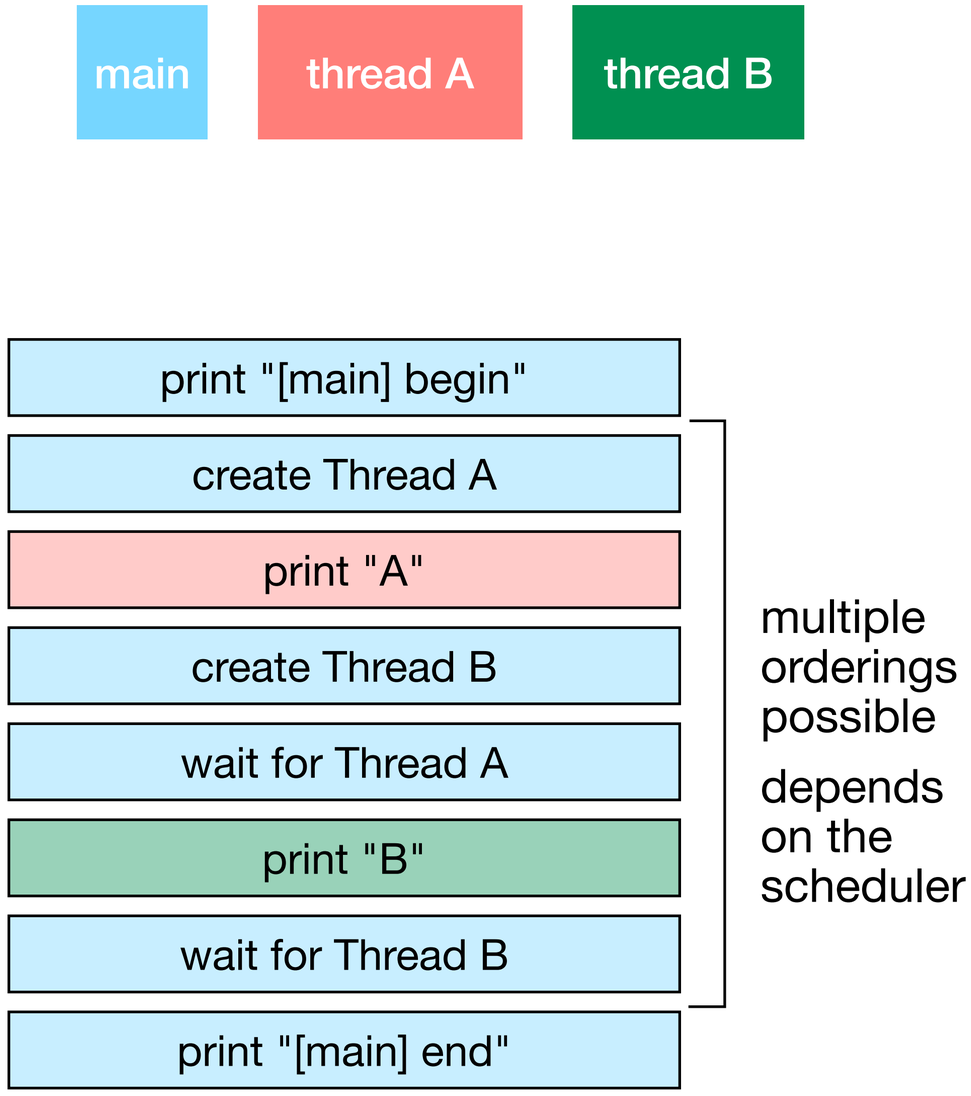
```c
// includes omitted
void *worker(void *arg) {
  printf("[thread] %s\n", (char *) arg);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p1, p2;
  int rc;
  printf("[main] begin\n");
  pthread_create(&p1, NULL, worker, "A");
  pthread_create(&p2, NULL, worker, "B");
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] end\n");
  return 0;
}
```

thread_print.c



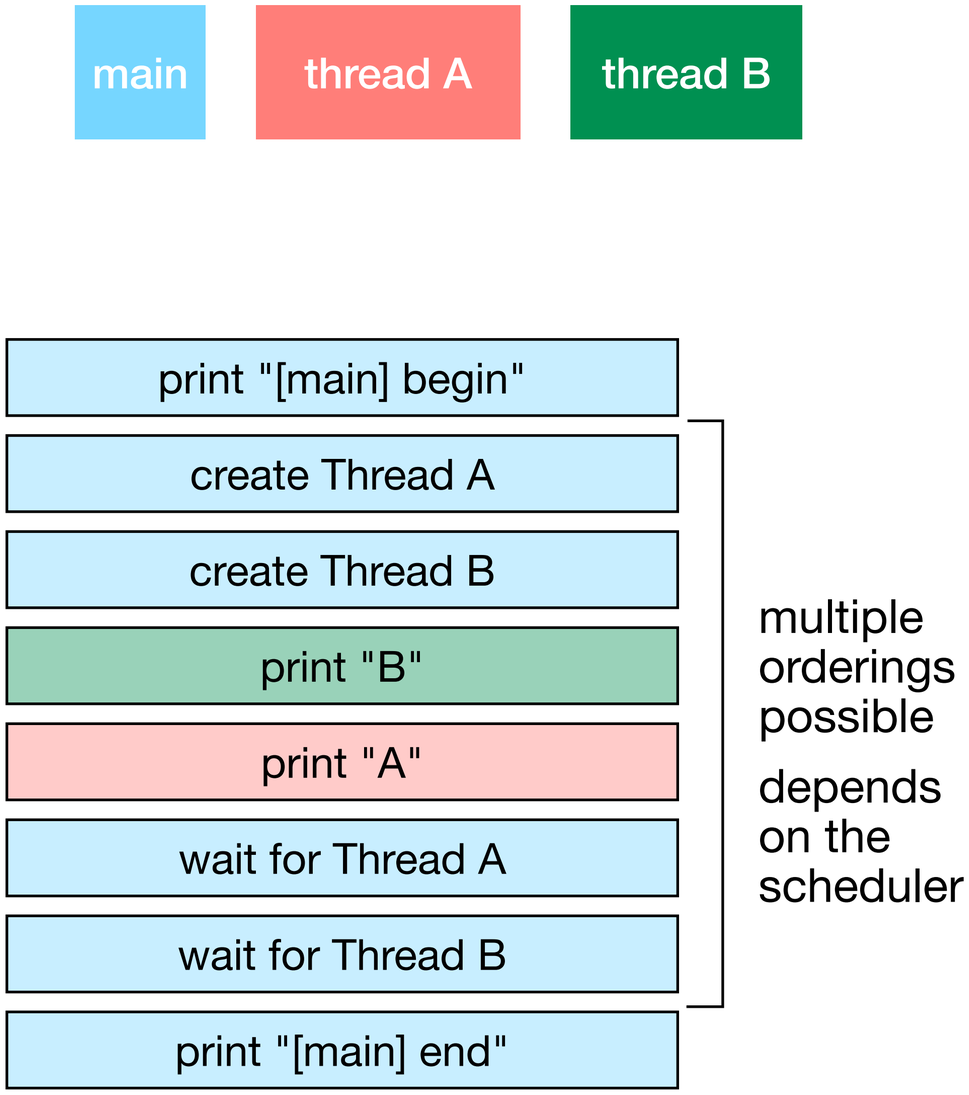
```c
// includes omitted
void *worker(void *arg) {
  printf("[thread] %s\n", (char *) arg);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p1, p2;
  int rc;
  printf("[main] begin\n");
  pthread_create(&p1, NULL, worker, "A");
  pthread_create(&p2, NULL, worker, "B");
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] end\n");
  return 0;
}
```

thread_print.c



```c
// includes omitted
void *worker(void *arg) {
  printf("[thread] %s\n", (char *) arg);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p1, p2;
  int rc;
  printf("[main] begin\n");
  pthread_create(&p1, NULL, worker, "A");
  pthread_create(&p2, NULL, worker, "B");
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] end\n");
  return 0;
}
```

thread_print.c



options are constrained by execution in main, e.g. A cannot run before it's created, B cannot be joined before A, etc.

```
// includes omitted
static volatile int counter = 0;
void *worker(void *arg) {
  int i;
  for (i = 0; i < 100000; i++)
    counter += 1;
 printf("[thread] %s\n", (char *) arg);
 return NULL;
}

int main(int argc, char *argv[]) {
 pthread_t p1, p2;
 int rc;
 printf("[main] begin\n");
 pthread_create(&p1, NULL, worker, "A");
 pthread_create(&p2, NULL, worker, "B");
 pthread_join(p1, NULL);
 pthread_join(p2, NULL);
 printf("[main] end counter = %d\n", counter);
 return 0;
}
```

thread_count.c

worker runs 100K times and increments a global counter

then prints its argument

this part is the same as before

```c
// includes omitted
static volatile int counter = 0;
void *worker(void *arg) {
  int i;
  for (i = 0; i < 100000; i++)
    counter += 1;
 printf("[thread] %s\n", (char *) arg);
 return NULL;
}

int main(int argc, char *argv[]) {
 pthread_t p1, p2;
 int rc;
 printf("[main] begin\n");
 pthread_create(&p1, NULL, worker, "A");
 pthread_create(&p2, NULL, worker, "B");
 pthread_join(p1, NULL);
 pthread_join(p2, NULL);
 printf("[main] end counter = %d\n", counter);
 return 0;
}
```

thread_count.c

```
>>> ./thread_print
[main] begin
[thread] A
[thread] B
[main] end counter = 136779
```

```
>>> ./thread_print
[main] begin
[thread] A
[thread] B
[main] end counter = 133750
```

we expected 200,000 but every time the answer is != 200,000 and **different**

why? **counter += 1 is not atomic**

```
counter = counter + 1
```

this is not an atomic operation

we can use a disassembler to examine the assembly code

address of the counter var

accumulator register

this is just 1

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

move the current value of the counter to the register

increment register by 1

save it back to the original address

```
counter = counter + 1
```

thread A

counter = 50

thread B

```
mov 0x8049a1c, %eax

add $0x1, %eax

mov %eax, 0x8049a1c
```

```
counter = counter + 1
```

thread A

counter = 50

thread B

eax = 50

```
mov 0x8049a1c, %eax

add $0x1, %eax

mov %eax, 0x8049a1c
```

A starts first

loads counter value into its register

```
counter = counter + 1
```

thread A

counter = 50

thread B

eax = 51

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

A starts first

increments the value in the register

```
counter = counter + 1
```

thread A

eax = 51

A is interrupted and put back into the ready queue

counter = 50

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

thread B

eax = 50

B is scheduled and starts running

loads counter value into its register

```
counter = counter + 1
```

thread A

eax = 51

A is interrupted and put back into the ready queue

counter = 50

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

thread B

eax = 51

B increments the value in its register

```
counter = counter + 1
```

thread A

eax = 51

counter = 51

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

thread B

eax = 51

B is interrupted and put back into the ready queue

A resumes

saves the contents of eax into counter

```
counter = counter + 1
```

thread A

eax = 51

A is finished

counter = 51

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

thread B

eax = 51

B is interrupted and put back into the ready queue

```
counter = counter + 1
```

thread A

eax = 51

counter = 51

```
mov 0x8049a1c, %eax

add $0x1, %eax

mov %eax, 0x8049a1c
```

thread B

eax = 51

B resumes

saves the contents of eax into counter

```
counter = counter + 1
```

thread A

eax = 51

counter = 51

thread B

eax = 51

```
mov 0x8049a1c, %eax

add $0x1, %eax

mov %eax, 0x8049a1c
```

both threads incremented the counter, but the final result was incremented only once

this is called a **race condition** - final results depend on the timing

critical section - code that can lead to race condition

```
counter = counter + 1
```

thread A

eax = 51

counter = 51

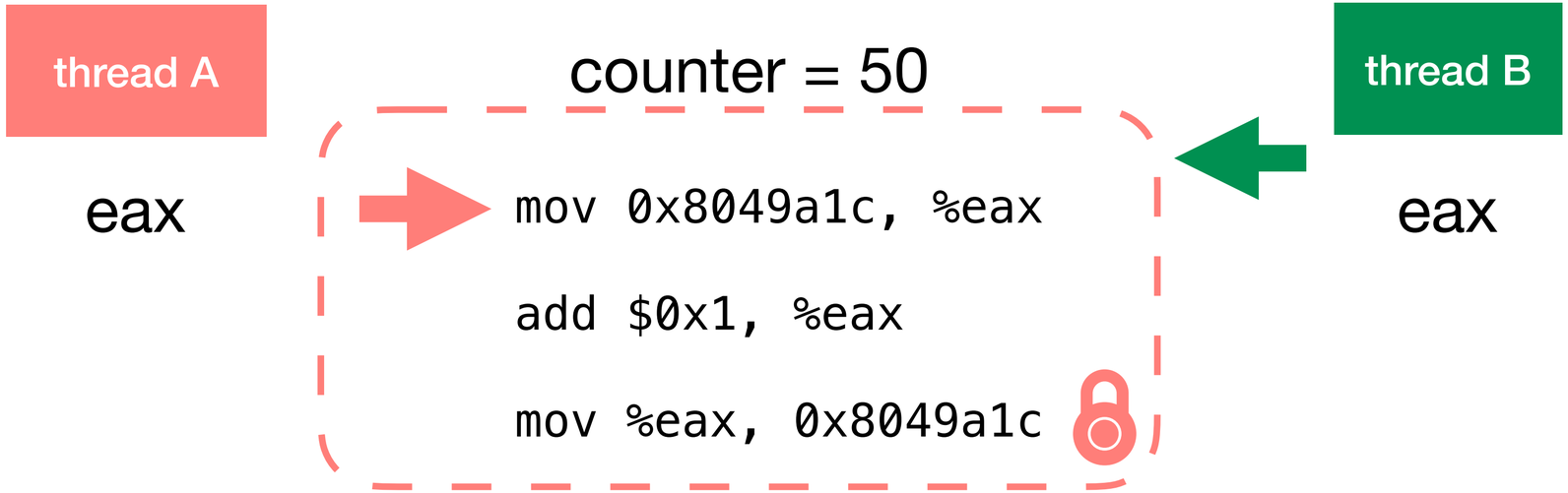
```
mov 0x8049a1c, %eax

add $0x1, %eax

mov %eax, 0x8049a1c
```

thread B

eax = 51

to make this work correctly, we need **mutual exclusion** - a guarantee that **only one thread** at a time can **execute** the critical section

```
counter = counter + 1
```



to make this work correctly, we need **mutual exclusion** - a guarantee that **only one thread** at a time can **execute** the critical section

Summary so far

- In most operating systems you have **little control over thread scheduling**
- You **cannot assume** that
  - threads will execute at the same rate
  - threads will execute in the order of creation
- **Race condition**
  - code will produce different results depending on the scheduling
    - multiple threads race to complete the task
    - order of execution is not guaranteed by the scheduler
- **Critical code section**
  - any non-atomic operation performed on shared data
  - can be potentially affected by concurrent execution

Pthread mutex

- Basic pair of pthread mutex functions
  - `pthread_mutex_lock(pthread_mutex_t *mutex)`
    - if the mutex is available, the thread will acquire it and continue
    - if the mutex is not available, the thread will block until it becomes available
    - if more than one thread is waiting for the lock, scheduler decides who will get it
  - `pthread_mutex_unlock(pthread_mutex_t *mutex)`
    - the lock owner will release the lock
    - another thread will block until it becomes available
- Put these around the critical code section to lock it up
  - guarantees that only the thread holding the lock can execute the code

static mutex initialization

declare as a global var

```
// includes omitted
pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;
static volatile int counter = 0;
void *worker(void *arg) {
  int i;
  for (i = 0; i < 100000; i++) {
    pthread_mutex_lock(&counter_mutex);
    counter += 1;
    pthread_mutex_unlock(&counter_mutex);
  }
 printf("[thread] %s\n", (char *) arg);
 return NULL;
}

int main(int argc, char *argv[]) {
 pthread_t p1, p2;
 int rc;
 printf("[main] begin\n");
 pthread_create(&p1, NULL, worker, "A");
 pthread_create(&p2, NULL, worker, "B");
 pthread_join(p1, NULL);
 pthread_join(p2, NULL);
 printf("[main] end counter = %d\n", counter);
 return 0;
}
```

lock the critical code section

the rest is the same as before

thread_count_lock.c

static mutex initialization

declare as a global var

```
// includes omitted
pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;
static volatile int counter = 0;
void *worker(void *arg) {
  int i;
  for (i = 0; i < 100000; i++) {
    pthread_mutex_lock(&counter_mutex);
    counter += 1;
    pthread_mutex_unlock(&counter_mutex);
  }
 printf("[thread] %s\n", (char *) arg);
 return NULL;
}

int main(int argc, char *argv[]) {
 pthread_t p1, p2;
 int rc;
 printf("[main] begin\n");
 pthread_create(&p1, NULL, worker, "A");
 pthread_create(&p2, NULL, worker, "B");
 pthread_join(p1, NULL);
 pthread_join(p2, NULL);
 printf("[main] end counter = %d\n", counter);
 return 0;
}
```

lock the critical code section

the rest is the same as before

thread_count_lock.c

```
>>> ./thread_print
[main] begin
[thread] A
[thread] B
[main] end counter = 200000
```

```
>>> ./thread_print
[main] begin
[thread] B
[thread] A
[main] end counter = 200000
```

```
// includes omitted
pthread_mutex_t counter_mutex;
static volatile int counter = 0;
void *worker(void *arg) {
  int i;
  for (i = 0; i < 100000; i++) {
    pthread_mutex_lock(&counter_mutex);
    counter += 1;
    pthread_mutex_unlock(&counter_mutex);
  }
  printf("[thread] %s\n", (char *) arg);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p1, p2;
  int rc;
  rc = pthread_mutex_init(&counter_mutex, NULL);
  if (rc) {fprintf(stderr, "pthread_mutex_init failed: %d\n", rc); return 1;}
  printf("[main] begin\n");
  pthread_create(&p1, NULL, worker, "A");
  pthread_create(&p2, NULL, worker, "B");
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] end counter = %d\n", counter);
  pthread_mutex_destroy(&counter_mutex);
  return 0;
}
```

this is unitialized now

dynamic lock initialization

this is the same as before

need to initialize mutex in main

always check if it worked

need to destroy the lock when finished

```c
// includes omitted
pthread_mutex_t counter_mutex;
static volatile int counter = 0;
void *worker(void *arg) {
  int i;
  for (i = 0; i < 100000; i++) {
    pthread_mutex_lock(&counter_mutex);
    counter += 1;
    pthread_mutex_unlock(&counter_mutex);
  }
  printf("[thread] %s\n", (char *) arg);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p1, p2;
  int rc;
  rc = pthread_mutex_init(&counter_mutex, NULL);
  if (rc) {fprintf(stderr, "pthread_mutex_init failed: %d\n", rc); return 1;}
  printf("[main] begin\n");
  pthread_create(&p1, NULL, worker, "A");
  pthread_create(&p2, NULL, worker, "B");
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] end counter = %d\n", counter);
  pthread_mutex_destroy(&counter_mutex);
  return 0;
}
```

this is unitialized now

dynamic lock initialization

this is the same as before

```
>>> ./thread_print
[main] begin
[thread] A
[thread] B
[main] end counter = 200000
```

```
>>> ./thread_print
[main] begin
[thread] B
[thread] A
[main] end counter = 200000
```

need to initialize mutex in main

always check if it worked

need to destroy the lock when finished

Condition variables

- Each condition variable is associated with the lock
  - `pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)`
    - puts the calling thread to sleep
    - unlocks the mutex
    - waits for the other thread to signal it
    - when signalled, reacquires the lock (may need to wait)
- When something changes about the condition
  - `pthread_cond_signal(pthread_cond_t *cond)`
    - unblocks at least one thread blocked by condition
    - if more than one thread is blocked, the order is decided by the scheduler

```
// includes omitted
pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t condition = PTHREAD_COND_INITIALIZER;

int ready = 0;

void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  printf("[waiter] got the lock\n");
  printf("[waiter] waiting for the ready\n");
  while (!ready) pthread_cond_wait(&cond, &mut);
  printf("[waiter] ready flag set!\n");
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}

void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  printf("[signaler] set ready flag = 1\n");
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  printf("[signaler] released mutex and signaled\n");
  return NULL;
}
```

initialize a mutex and condition variable

this is the variable used for the condition

thread will not start until ready is set to 1

then will do some work and release the lock

the other thread will acquire the lock and set ready to 1

then will signal to the other thread via cond variable

and release the lock

t_waiter will wait for the t_signaler to set the global variable ready=1 to start work

main is the same as before, just creates two threads and then joins them

```
int main(int argc, char *argv[]) {
  pthread_t t_waiter, t_signaler;
  int rc;
  pthread_create(&t_waiter, NULL, waiter, NULL);
  pthread_create(&t_signaler, NULL, signaler, NULL);
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] program finished\n");
  return 0;
}
```

```c
// includes omitted
pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t condition = PTHREAD_COND_INITIALIZER;

int ready = 0;

void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  printf("[waiter] got the lock\n");
  printf("[waiter] waiting for the ready\n");
  while (!ready) pthread_cond_wait(&cond, &mut);
  printf("[waiter] ready flag set!\n");
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}

void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  printf("[signaler] set ready flag = 1\n");
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  printf("[signaler] released mutex and signaled\n");
  return NULL;
}
```

```
>>> ./thread_cond
[waiter] got the lock
[waiter] waiting for the ready
[signaler] got the lock
[signaler] set ready flag = 1
[signaler] released mutex and signaled
[waiter] ready flag set!
[main] program finished
```

```c
int main(int argc, char *argv[]) {
  pthread_t t_waiter, t_signaler;
  int rc;
  pthread_create(&t_waiter, NULL, waiter, NULL);
  pthread_create(&t_signaler, NULL, signaler, NULL);
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("[main] program finished\n");
  return 0;
}
```

thread A

ready = 0

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

both threads start to execute their functions

whichever gets to the lock first, will acquire it
(this depends on the scheduler)

thread A

ready = 0

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

let's say A was first, it gets the lock and continues to execute

B is now blocked until mutex is released by A

thread A

ready = 0

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

let's say A was first, it gets the lock and continues to execute

B is now blocked until mutex is released by A

thread A

ready = 0

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

A checks if ready is set

ready is still 0 so pthread_cond_wait is called next

thread A

ready = 0

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

A checks if ready is set

ready is still 0 so pthread_cond_wait is called next

A is now blocked and mut lock is released

thread A

ready = 0

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

B now acquires the lock (may need to wait more depending on the scheduler)

thread A

ready = 1

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

B now acquires the lock (may need to wait more depending on the scheduler)

B sets ready=1

thread A

ready = 1

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

B now acquires the lock (may need to wait more depending on the scheduler)

B sets ready=1
signals to all threads blocked by cond

thread A

ready = 1

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

B now acquires the lock (may need to wait more depending on the scheduler)

B sets ready=1
signals to all threads blocked by cond
releases the mutex and exits

thread A

ready = 1

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

A wakes up (depends on the scheduler)

acquires the lock

thread A

ready = 1

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

A wakes up (depends on the scheduler)

acquires the lock

and finishes the rest

thread A

ready = 1

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

A wakes up (depends on the scheduler)

acquires the lock

and finishes the rest

thread A

ready = 1

thread B

```
void *waiter(void *arg) {
  pthread_mutex_lock(&mut);
  while (!ready) pthread_cond_wait(&cond, &mut);
  do_work();
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

```
void *signaler(void *arg) {
  pthread_mutex_lock(&mut);
  ready = 1;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mut);
  return NULL;
}
```

A wakes up (depends on the scheduler)

acquires the lock

and finishes the rest

```
int balance = 0;

void* worker(void* arg) {
    balance++;
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;

    pthread_create(&p, NULL, worker, NULL);

    balance++;

    pthread_join(p, NULL);

    printf("Balance=%d\n", balance);

    return 0;
}
```

any problems here?

both the main thread and worker thread increment balance -> race condition

how to fix?

put locks around both balance increments

```
void *t1(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&m1);
    usleep(100000);
    pthread_mutex_lock(&m2);
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}
```

t1 acquires m1 lock

takes a short nap

t1 wakes up and starts waiting for m2 to be released

this is the same code as before, just creates two locks, two threads and waits for them to finish

except they never will…

these two threads are now **deadlocked**

```
void *t2(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&m2);
    pthread_mutex_lock(&m1);
    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    return NULL;
}
```

second thread locks m2

and starts waiting for m1 to be released

```
pthread_mutex_t m1 =
PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 =
PTHREAD_MUTEX_INITIALIZER;
int main(void)
{
    pthread_t p1, p2;

    pthread_create(&p1, NULL, t1, NULL);
    pthread_create(&p2, NULL, t2, NULL);

    pthread_join(p1, NULL);
    pthread_join(p2, NULL);

    return 0;
```

Thread API guidelines

- Keep it simple - code to lock or signal should be as simple as possible
- Minimize thread interactions
  - Always use condition variables to signal between threads
- Initialize locks and condition variables - undefined behavior otherwise
- Beware of mutex deadlocks
- Check return codes - most threads/systems API functions fail quietly and then bugs crop up in other places
- Do not pass stack variables between threads or to the main function
- Read the manual

Summary

- **Scheduling threads**
  - depends on the multithreading model
    - can be scheduled as a group or individually
  - difficult to predict when each thread will run -> **do not make assumptions**
- **Race condition** - when order of thread execution changes the results
- **Critical section** - part of the code that can be affected by race condition
  - usually because the operations are not atomic
- **Mutex lock** - protects the critical section from concurrent execution
- **Conditional variable** - allows signalling to other threads that are waiting for a condition to be satisfied

Reading

- Chapter 27 Interlude: Thread API
  - https://pages.cs.wisc.edu/~remzi/OSTEP/threads-api.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-intro
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-api
- Optional reading
  - Chapter 10 Multiprocessor scheduling
    - https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-multi.pdf
  - https://cs.stanford.edu/people/karpathy/advice.html

Quiz 12

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999